Roll No. _____

Name: _____

Seat: _____

Data Structures and Algorithms
CS213 (Autumn 2026)

12 September 2026 - Mid-Semester Examination

Instructor: Prof. Manoj Prabhakaran

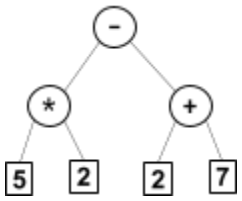
Total: 50 points | *Duration: 120 min*

A. Short Answer Questions [15 pts]

1. Suppose f, g are two positive real-valued functions such that $f(n) = O(n)$, $g(n) = O(\log n)$. Then which of the following is/are correct: [2 pts]

   - ☑ $f(n) + g(n) = O(n)$
   - ☐ $f(n)/\, g(n) = O(n)$
   - ☐ $f(n)\, g(n) = O(n)$
   - ☑ $|f(n) - g(n)| = O(n)$

2. An arithmetic expression can be represented by a binary tree (called the parse tree) whose leaves are operands and whose internal nodes are operators. For instance, the parse tree shown here corresponds to the expression `5*2 - (2+7)`.



   Which traversal of a parse tree outputs the expression in Reverse Polish Notation? [1 pt]

   - ☐ Pre-order
   - ☐ In-order
   - ☑ Post-order
   - ☐ Breadth-first

   Write down the expression `5*2 - (2+7)` in Reverse Polish Notation: [1 pt]

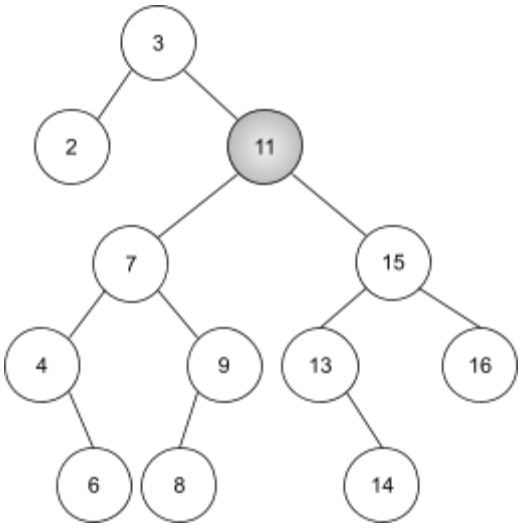
   `5 2 * 2 7 + -`

   1 point total if "Polish Notation" (instead of RPN) used consistently: Pre-order chosen and answer above is - * 5 2 + 2 7.

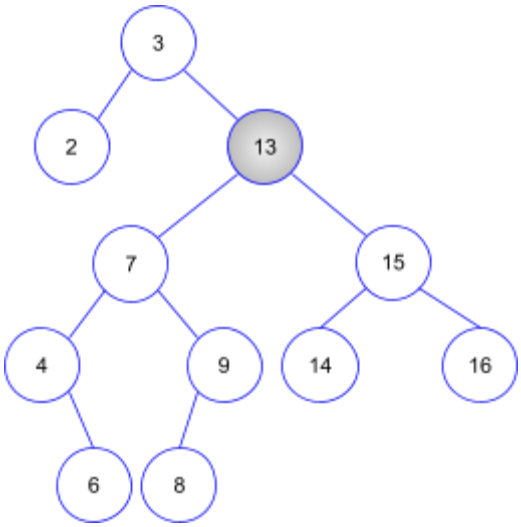
3. Consider the recursive function (from the lectures) that prints the pre-order sequence of a nearly-complete binary tree with n nodes. The time and additional space it takes are (choose the option with the tightest bounds): [1 pt]

- ☐ O(n) time, O(1) additional space
- ☐ O(log n) time, O(log n) additional space
- ☑ O(n) time, O(log n) additional space
- ☐ O(log n) time, O(1) additional space

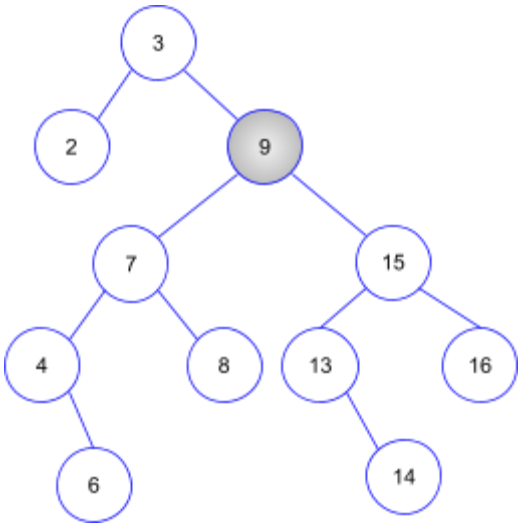
4. Suppose a Binary Search Tree is in the state shown below (only internal nodes are shown), and we delete the shaded node with the number 11, using the method from class. Draw the state of the tree after this deletion. [2 pts]



Either of the following gets full 2 pts.
1 pt if there is a single error in some node value.



or



5. The array `[2,5,3,9,7,4,8]` represents a min-heap. Suppose we insert an element 1 into this heap. Then the array becomes: [2.5 pts]

[1, 2, 3, 5, 7, 4, 8, 9]

1 point if a valid heap with correct elements: e.g. [1 2 3 4 5 7 8 9]

6. Suppose an operation F is such that F(k) costs k units if k is a power of two, and 1 unit otherwise. The total cost of invoking F on 1, …, n is [2 pts]

- ☐ Θ(log n)
- ☑ Θ(n)
- ☐ Θ(n log n)
- ☐ Θ(n^2)

Hence the amortised cost per operation is Θ(__**1**____) . [0.5 pt]

7. Consider using a deque to find the maximum element in each window of size N, in a stream of T integers, as discussed in class. Fill in the blanks below. [3 pts]

For the first blank, choose a relation among <=, >= and ==. Fill in each of the next two blanks using either "front" *or* "back."

Before pushing a new item into the front of the deque, all elements_____**<=**_____ it are popped from the _____**front**_____ of the deque; also if there is an element that is too old to be in the window of size N, it is popped from the _____**back**_____ of the deque.

Processing a single new element in this manner takes time O(___**N**_____) and processing all T elements takes time Θ(____**T**____). **0.5 pts for the first 4 blanks, 1 pt for the last.**

OK if the first blank filled in as < (instead of <=).

B. Binary Search [5 pts]

Complete the function below so that it returns the index of the largest element in A that is **strictly smaller than** x, or if no such element exists, returns −1. A is guaranteed to be a sorted vector (in ascending order) with **distinct** integers.

```
int predecessor(const vector<int>& A, int x) {
    if (A.empty()) return -1;
    int left = 0, right = A.size()-1;
    while (left <= right) {
        int mid = (left + right)/2;
        if (A[mid] < x)
            _____left = mid + 1;_____;     1.5 pt
        else
            _____right = mid - 1;_____;     1.5 pt
    }
    return right;
}
```

The worst-case running time of this algorithm is **1 pt**

- ☐ Θ(1)
- ☑ Θ(log n)
- ☐ Θ(n)
- ☐ Θ(n log n)

and the additional space used (beyond the data) is **1 pt**

- ☑ Θ(1)
- ☐ Θ(log n)
- ☐ Θ(n)
- ☐ Θ(n log n)

C. Circular Array Queue [8 pts]

In this problem, we consider a fixed-capacity queue of characters, implemented using a circular array, as in class. The implementation uses an array of `N` characters and variables `front`, `back`, and `is_empty`.

C1. Trace the Execution

Suppose, the array is of size `N = 6`, and at a certain point its contents are:

Index	0	1	2	3	4	5
Content	-	-	c	b	a	-

and the other variables are set to: `front = 4, back = 2, is_empty = false.`
So at this point, the queue contained, from front to back: `[ a, b, c ]`. Then the following operations are performed in order:

```
enqueue('d'); dequeue(x); enqueue('e'); enqueue('f');
```

Fill in the state of the array and the variables after these operations are performed. When an item in a cell is not part of the queue, show it as '-'.

Index	0	1	2	3	4	5
Content	**e**	**d**	**c**	**b**	**-**	**f**

2.5 pt (2 pt if everything else is correct, but 'a' instead of '-')

`front = __3___, back = __5___, is_empty = __false____.` **0.5 pt per blank**

C2. Complete the Code

Complete the code below for the dequeue function in this implementation. Your implementation should be consistent with the example above.

```
// helper functions to use as needed
int dec(int i) { return i==0 ? N-1 : i-1; }
int inc(int i) { return i==N-1 ? 0 : i+1; }

bool dequeue(char& x) {
    if (is_empty) return false;

    x = Q[front];

    if (front == back)

        _____is_empty = true_____;  2 pts

    else

        _____front = dec(front)_____;  2 pts

    return true;
}
```

For the second blank: 1.5 pt for “dec(front);” (without assignment).
1 pt for front--, --front, or front = front - 1.
0.5 pt for “front=inc(front);” or “inc(front);” or front++ etc.

D. r-th Smallest Element in k Sorted Lists [10 pts]

You are given k non-empty singly linked lists containing altogether n nodes, of the following type.

```
struct node { int val; node* next; };
```

Each of these lists is sorted in ascending order of `val` in the nodes. We shall assume that the values are distinct across all n nodes. Our goal in this problem is to find the node with the rth smallest element in the union of all k lists ($1 \le r \le n$). Towards this, we will use a heap data structure `minNodeHeap` which stores `node*` pointers, with the node having the smallest `val` at the top. It supports the following operations:

```
void push(node* p);   // insert p
node* top();          // return a pointer to the node with minimum val
void pop();           // remove the node with minimum val
```

When the heap contains at most k nodes, `push()` and `pop()` take O(log k) time, and `top()` takes O(1) time.

Consider the following procedure:

1) Start with an empty heap (of minNodeHeap type).
2) For every non-empty list, insert (a pointer to) its first node into the heap.
3) Repeat r times:
    i) remove the minimum-valued node from the heap;
    ii) if that node has a successor in its own list, insert the successor into the heap.
4) Report the last node removed from the heap as the one containing the rth smallest element.

D1. Trace the Procedure

Consider k = 3 lists:

```
L1:  2 -> 8 -> 12
L2:  1 -> 5 -> 14
L3:  4 -> 7 -> 10
```

Let r = 5. Execute the above procedure and fill in the blanks/cells below for the iteration of the loop (step 3) in the procedure.

Values of the nodes in the heap after step (2): ___**2, 1, 4**____ (in any order). **0.5 pts**

Iteration of the loop in step (3)	Value removed from heap in step (i)	Value inserted into the heap in step (ii), if any
1	**1**	**5**
2	**2**	**8**
3	**4**	**7**
4	**5**	**14**
5	**7**	**10**

2 pts (1.5 pt if the first 3 rows are correct. 0.5 pt if the first 3 removals are correct, but the insertions are not all correct.)

The 5th smallest value is _____**7**_____. **0.5 pts**

D2. Complete the Code

Complete the function below that implements the procedure. Fill in each blank using a single expression or statement.

```
node* rth_smallest(const vector<node*>& heads, int r) {
    minNodeHeap H;                                          // step (1)

    for (node* p : heads) {                                 // step (2)
        if (p != nullptr)

            ____H.push(p);_____;  0.5 pt  (0.25 if "push(p)" appears)
    }
    node* p = nullptr;
    for (int i = 0; i < r; ++i) {                           // step (3)

        p = ___H.top();_________;    0.5 pt  (0.25 if "top" appears)
        H.pop();

        if (_p->next != nullptr_)  (or just "p->next")          1 pt

            ____H.push(p->next)_____;                           1 pt
    }

    return __p__;    1 pt (0.5 pt if "p->val")              // step (4)
}
```

D3. Analyse the Code

a) At any point, the heap contains at most __k____ nodes. **0.5 pt**

b) The worst-case running time of the above procedure is: **1 pt**

- ☐ Θ(r log k)
- ☑ Θ((k+r) log k)
- ☐ Θ(r log n) where n is the total number of nodes across all lists
- ☐ Θ(kr)

c) At the end of step (2) and at the end of each iteration of step (3), we have at most one element from each list in the heap. Select all the correct statements below that explain why the algorithm is designed this way?
+0.75 for selecting each correct option,
-0.5 for selecting each incorrect option

- ☐ Inserting two nodes from the same list would violate the heap property.
- ☑ A second node from a list is not useful in the heap as it does not affect the minimum.
- ☐ Since the heap was initialized with k nodes, it cannot ever contain more than k nodes.
- ☑ All n elements can be added to the heap upfront: while remaining correct, it can be slower.
- ☐ Inserting two or more nodes from the same list will result in a wrong answer.

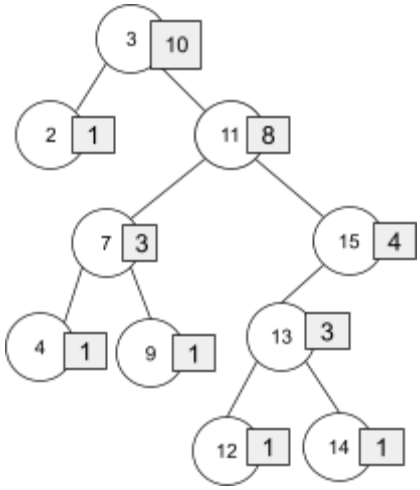
E. Rank in an Augmented Binary Search Tree [12 pts]

In a Binary Search Tree (BST) with **distinct** keys, the **rank** of a key is defined as its position in the sorted order of all the keys, starting from rank 1. Thus, the smallest key in the BST has rank 1. In this problem we augment BSTs to efficiently compute the rank of any given key.

For this, the node structure is augmented as follows:

```
struct node {
  int key, subsize;
  node *left, *right, *parent;
};
```

where `subsize` records the number of internal nodes in the subtree rooted at that node (counting the node itself). The illustration shows a binary search tree with the `key` (in the circle) and the `subsize` (in the square) fields at each internal node.



We use the function below to extend the definition of subtree-sizes to empty trees (nullptr):

```
int subsize(node* p) { return (p == nullptr) ? 0 : p->subsize; }
```

E1. An Example

What is the rank of the key 13 in the above tree? __8__. **1 pt**

For a key in the tree, we will call a node *important* for that key, if it is in the path from the root to that key, but its left child is not in that path. (Note that a key's own node is important for itself.)

List all the important nodes for 13: ____3, 11, 13____. **1 pt. any order OK.**

Sum up the subtree sizes of the left children of all the important nodes for 13: __5__. **1 pt.**

Complete the formula: `rank(key)` = $\sum_{p \in imp(key)}$ (`subsize(p->left)` + ___1___)
where `imp(key)` is the set of (pointers to) all important nodes for `key`. **1 pt.**

E2. Complete Code for `rank( )`

The function `rank( )` below will be added to a BST data structure, to search for a key and compute its rank. If the key is not present in the tree it returns `0`. Fill in each blank using at most one expression or statement.

```
int tree::rank(int x) {
    node* p = root;
    int r = 0;
    while (p != nullptr) {
        if (x < p->key) {

            ___p = p->left_____; 1 pt.
        }
        else if (p->key < x) {

            r += __subsize(p->left) + 1_; 1 pt. -0.5 for omitting '+ 1',

-0.25 for p->left->subsize instead of subsize(p->left).

            _p = p->right_____; 1 pt.
        }
        else {
☐           return ___r + subsize(p->left) + 1___;  1 pt. 0.5 for
  just r. 0.75 for "r + <expression from blank 2 above>".
        }
    }
    return 0;
}
```

E3. Analysis

1. On entering the `while` loop, which statement best describes `r`? **1 pt.**

- ☐ The number of nodes already visited during the search.
- ☑ The number of keys smaller than every key in the subtree rooted at `p`.
- ☐ The number of keys smaller than `p->key`
- ☐ The number of keys in the left subtree of `p`.

2. Suppose a new node is inserted into the BST using the BST insertion algorithm from class. Answer the following.

   a. The `subsize` in the new node should be set to: **1 pt.**

   Answer: ____1_____

   b. For every ancestor of the new node, its `subsize` should be: **1 pt.**

   - ☑ increased by 1.
   - ☐ decreased by 1.
   - ☐ doubled.
   - ☐ left unchanged.

   c. Updating all affected `subsize` fields takes (choose the tightest bound): **1 pt.**

   - ☐ O(1)
   - ☐ O(log n) for every BST
   - ☑ O(h), where h is the height of the tree
   - ☐ O(n) for every BST

Bonus Problem: Incremental Resizing of a Dynamic Array

A dynamic array normally has an occasional expensive `push_back`: when the array becomes full, all its elements are copied into a new array of twice the capacity. In this problem we consider an alternate strategy. After allocating the larger array, instead of copying all the elements at once, **a single element is migrated from the old array to the new one during that and each subsequent `push_back`** until migration is complete.

Our new data structure is as follows (with two functions to be implemented):

```
struct dynamicArray {
    int* old_array = nullptr;     // not null when migration in progress
    int* current_array = new int[1];          // initially capacity is 1

    int capacity = 1;   // capacity of current_array
    int size = 0;       // number of elements currently stored
    int copied = 0;     // number of old elements already copied

    void push_back(int x);   // TODO
    int& at(int i);          // TODO

    void begin_migration() {
        old_array = current_array;
        capacity *= 2;
        current_array = new int[capacity];
        copied = 0;
    }
    void end_migration() {
      delete[] old_array;
      old_array = nullptr;
    }
    bool pop_back(int& x) {
      if (size == 0) return false;
      x = at(size-1); --size;
      if (old_array != nullptr && copied==size) end_migration();
      return true;
    }
};
```

When no copying is in progress, `old_array == nullptr`.
When copying is in progress:

- `current_array` has capacity `capacity` and `old_array` has capacity `capacity/2`;
- entries `0, ..., copied-1` have already been copied from `old_array` into the same positions of `current_array` and will be read from/updated there.
- entries `copied, ..., min(size,capacity/2)-1` are still stored in `old_array`;
- `at(i)` checks if index `i` is stored in `current_array` or `old_array` and returns a reference to the correct location.

Bonus-1. Code for push_back( )

Complete the function below.

```
void dynamicArray::push_back(int x) {
    //---- If full, start migrating to an array of twice the capacity.
    if (size == capacity)
       begin_migration();
    //---- If migration is in progress, migrate one old element.
    if (old_array != nullptr) {

        current_array[_______________] = _____________________________;
        ++copied;
        // Once all old elements have been copied, discard old_array.
        if (copied == capacity/2 || copied == size)
           end_migration();
    }
    //---- Finally insert the new element
    ++size;
    at(__________________) = x;
}
```

Bonus-2. Code for `at()`

Complete the function `at()` below. You may assume `0 <= i < size`.

```
int& dynamicArray::at(int i) { // assumes  0 <= i < size
    // unmigrated items should be accessed in old_array
    if (old_array != nullptr
       && _____________________________ && _____________________________)
        return old_array[i];
    return current_array[i]; //if not from old_array, from current_array
}
```

Bonus-3. Analysis

Suppose `begin_migration()` is called from `push_back()` when `size` is M.

When `begin_migration()` returns, `capacity is` _____.

Then, at least _____ `push_back` operations (including the one that triggered migration) must occur before `size == capacity`.

In each of these `push_back` operations, until migration is complete, one item is copied from `old_array` to `current_array`. Based on this analysis and the code above, **select all the statements** that are guaranteed to hold (irrespective of the order in which the functions in `push_back()`, `pop_back()` and `at()` are called). Assume that allocating or deallocating an uninitialized integer array of any length takes O(1) time.

- ☐ Each call to `push_back()`, `pop_back()` and `at()` takes O(1) worst-case time.
- ☐ After n `push_back()` calls, the maximum amount of memory used by the data structure is the memory required for 2n + O(1) items.
- ☐ At the point where `begin_migration()` is called from `push_back()`, `old_array` is necessarily `nullptr`.
- ☐ At the point where `end_migration()` is called from `push_back()`, if `copied == capacity/2`, then `size` equals `capacity`.

SCRATCH SPACE —---------- WILL NOT BE GRADED

SCRATCH SPACE —---------- WILL NOT BE GRADED

SCRATCH SPACE —---------- WILL NOT BE GRADED

SCRATCH SPACE —---------- WILL NOT BE GRADED